

# Microsoft AutoGen Class Assignments

## Assignment 0: UserProxyAgent Integration in SoM Teams

Due Date: August 9th, 2025

### Objective

Demonstrate your understanding of human-in-the-loop implementation using UserProxyAgent in Microsoft AutoGen's Society of Mind (SoM) framework for both inner and outer team configurations.

### Requirements

#### Part A: Inner Team Integration (50 points)

1. **Create a multi-agent inner team** with at least 3 specialized agents
2. **Integrate UserProxyAgent** to allow human intervention at critical decision points
3. **Implement human feedback loops** where the user can:
  - Approve/reject agent recommendations
  - Provide additional context or constraints
  - Override agent decisions when necessary

#### Part B: Outer Team Integration (50 points)

1. **Design an outer team structure** that coordinates multiple inner teams
2. **Position UserProxyAgent strategically** to manage inter-team communication
3. **Enable human oversight** of:
  - Team coordination decisions
  - Resource allocation between teams
  - Final output validation

### Deliverables

1. **Code Implementation** (80 points)
  - Working AutoGen code with proper UserProxyAgent integration
  - Clear documentation of agent roles and responsibilities
  - Demonstration of human-in-the-loop functionality
2. **Flow Diagram** (20 points)
  - Visual representation of your SoM architecture
  - Clear indication of UserProxyAgent placement
  - Human intervention points marked

### Evaluation Criteria

- Understanding of SoM concepts (25%)
- Proper UserProxyAgent implementation (35%)
- Code quality and documentation (25%)
- Creative problem-solving approach (15%)

---

## Assignment 1: Professional ATS Resume Scoring System

**Due Date: August 16th, 2025**

## **Objective**

Build a comprehensive Applicant Tracking System (ATS) resume scoring solution using Microsoft AutoGen that provides consistent scoring, improvement recommendations, and professional visualizations.

## **System Architecture Requirements**

### **Core Components**

#### **1. Resume Processing Agent**

- Parse resumes in multiple formats (PDF, DOCX, TXT)
- Extract key information (skills, experience, education, etc.)
- Convert to standardized JSON format

#### **2. ATS Scoring Agent**

- Implement consistent scoring algorithm
- Maintain scoring consistency across multiple runs
- Generate numerical scores with detailed breakdowns

#### **3. Job Description Analysis Agent**

- Parse and analyze job descriptions
- Identify key requirements and qualifications
- Create matching criteria for resume evaluation

#### **4. Improvement Recommendation Agent**

- Compare resume against job requirements
- Generate specific improvement suggestions
- Provide actionable feedback for score enhancement

#### **5. Visualization Agent**

- Create professional charts and graphs
- Generate comparison visualizations
- Produce exportable reports

## **Technical Requirements**

### **Database Integration**

- **Connect to external database for:**
  - Storing resume data and scores
  - Maintaining job description templates
  - Tracking scoring history and consistency
  - User session management
  - Suggest Open Relevant Jobs based on Resume from the Database

### **RAG (Retrieval-Augmented Generation) Implementation**

- **Knowledge Base Integration:**
  - Industry-specific requirements database
  - Best practices for resume optimization
  - Skill taxonomy and keyword matching
  - Historical scoring patterns

## Tools and External APIs

- **File Processing Tools:**
  - PDF extraction capabilities
  - Document format conversion
  - Text cleaning and preprocessing
- **Database Tools:**
  - CRUD operations for resume storage
  - Query tools for historical data
  - Backup and recovery mechanisms

## Functional Requirements

### 1. Resume Upload and Processing

- Support multiple file formats
- Automatic format detection and conversion
- Error handling for corrupted files
- Progress tracking for large files

### 2. ATS Score Generation

- **Consistency Guarantee:** Same resume should produce identical scores
- **Detailed Breakdown:** Scores by category (skills, experience, education, format)
- **Benchmark Comparison:** Compare against industry standards
- **Confidence Intervals:** Provide score reliability metrics

### 3. Job Description Integration

- Upload JD functionality
- Automatic requirement extraction
- Customizable matching criteria
- Industry-specific scoring adjustments

### 4. Improvement Recommendations

- **Text-based Suggestions:**
  - Missing keywords identification
  - Format improvement recommendations
  - Content structure optimization
  - Skill gap analysis
- **Visual Improvements:**
  - Before/after comparison charts
  - Skill match heatmaps
  - Score breakdown pie charts
  - Progress tracking dashboards

### 5. Streamlit Frontend

- **User Interface Requirements:**
  - Clean, professional design
  - Drag-and-drop file uploads
  - Real-time processing indicators
  - Interactive visualizations
  - Export functionality (PDF, CSV)

## Deliverables

### 1. Complete AutoGen System (60 points)

- All required agents implemented and integrated
- Working database connections
- RAG system functionality
- Consistent scoring mechanism

### 2. Streamlit Application (25 points)

- Professional user interface
- All required features implemented
- Responsive design
- Error handling and user feedback

### 3. Documentation (10 points)

- **Technical Documentation:**
  - System architecture diagram
  - API documentation
  - Database schema
  - Setup and installation guide
- **User Manual:**
  - Step-by-step usage instructions
  - Feature explanations
  - Troubleshooting guide

### 4. Demo and Testing (5 points)

- **Live Demonstration:** 10-minute presentation of full functionality
- **Test Cases:** Provide 5 sample resumes with expected scores
- **Consistency Testing:** Demonstrate score consistency across multiple runs

## Technical Specifications

### Scoring Algorithm Requirements

- **Weighted Categories:**
  - Skills Match (30%)
  - Experience Relevance (25%)
  - Education Alignment (15%)
  - Format and Structure (15%)
  - Keyword Optimization (15%)

### Performance Requirements

- Resume processing: < 30 seconds
- Score calculation: < 10 seconds
- Database queries: < 2 seconds
- UI responsiveness: < 3 seconds for all interactions

### Data Requirements

- Support resumes up to 5MB
- Handle concurrent users (minimum 10)
- Store processing history for analysis
- Maintain data privacy and security

**Bonus Features (Extra Credit - 10 points)**

- **AI-Powered Resume Rewriting:** Suggest specific text improvements
- **Batch Processing:** Upload and process multiple resumes
- **API Integration:** Connect to job boards for real-time JD fetching
- **Advanced Analytics:** Trend analysis and success rate tracking
- **Mobile Responsiveness:** Optimize for mobile devices

**Submission Guidelines**

**Code Submission**

- GitHub repository with clear README
- All dependencies listed in requirements.txt
- Docker containerization (optional but recommended)
- Environment variables properly configured

**Documentation**

- Include video walkthrough (5-10 minutes)
- Technical architecture diagram
- Performance testing results
- User feedback incorporation

**Evaluation Criteria**

| Component                | Weight | Description                                      |
|--------------------------|--------|--------------------------------------------------|
| Technical Implementation | 40%    | Code quality, architecture, AutoGen integration  |
| Functionality            | 30%    | Feature completeness, consistency, reliability   |
| User Experience          | 15%    | UI/UX design, ease of use, error handling        |
| Innovation               | 10%    | Creative solutions, bonus features, optimization |
| Documentation            | 5%     | Code documentation, user guides, technical specs |

**Resources and Support (Just Suggestion)**

**Recommended Libraries**

- **AutoGen:** Microsoft AutoGen framework
- **Streamlit:** Frontend development
- **SQLAlchemy:** Database ORM
- **PyPDF2/pdfplumber:** PDF processing
- **spaCy/NLTK:** Natural language processing
- **Plotly/Matplotlib:** Data visualization
- **python-docx:** Word document processing

**Database Options**

- **NoSQL:** Acceptable for development
- **MongoDB:** For document-based storage